

The background is a dark blue gradient with abstract white and light blue geometric patterns. On the left, there are several concentric circles with arrows indicating a clockwise direction. Some of these circles have numerical labels: 40, 150, 160, 170, 180, 190, 200, 210, 220, 230, 240, 250, and 260. There are also smaller circles and curved lines scattered across the background.

# INTRODUCTION AUX TRAITEMENTS DISTRIBUÉS AVEC SPARK ET HADOOP

ESGI - 5EME ANNÉE - ARCHITECTURE LOGICIEL

PRÉSENTÉ PAR YIDHIR MOUDOUB  
YIDHIR.MOUDOUB@OUTLOOK.COM

## Objectifs du cours

- Comprendre les concepts fondamentaux du traitement distribué.
- Découvrir l'écosystème Hadoop et ses composants principaux.
- Manipuler Apache Spark pour le traitement efficace de données à grande échelle.
- Être capable de choisir la bonne technologie selon un cas d'usage Big Data.

## Évaluation

- 40% : Exercices pratiques
- 60% : Mini-projet final (code + présentation orale)

 **Plan sur 5 séances de 3 heures**

- **Séance 1 - Introduction aux systèmes distribués et à Hadoop (3h)**
- **Séance 2 - Limitations de MapReduce et introduction à Apache Spark (3h)**
- **Séance 3 - Traitement structuré avec Spark SQL et DataFrames (3h)**
- **Séance 4 - Traitement temps réel avec Spark Streaming (3h)**
- **Séance 5 – Projet pratique et présentations orales (3h)**

 **Plan sur 5 séances de 3 heures**

## **Séance 1 - Introduction aux systèmes distribués et à Hadoop (3h)**

### **Théorie (1h30)**

- **Notions de base des systèmes distribués**
  - Définition, enjeux, caractéristiques
  - Scalabilité, tolérance aux pannes, latence
- **Le modèle MapReduce**
- **Présentation de l'écosystème Hadoop :**
  - Stockage distribué : HDFS
  - Calcul distribué

### **Pratique (1h30)**

- **Calculs arithmétiques à l'aide du Map Reduce**
- **WordCount avec le paradigme du Map Reduce**

 **Plan sur 5 séances de 3 heures**

## **Séance 1 - Introduction aux systèmes distribués et à Hadoop (3h)**

### **Théorie (1h30)**

- **Notions de base des systèmes distribués**
  - Définition, enjeux, caractéristiques
  - Scalabilité, tolérance aux pannes, latence
- **Le modèle MapReduce**
- **Présentation de l'écosystème Hadoop :**
  - Stockage distribué : HDFS
  - Calcul distribué

### **Pratique (1h30)**

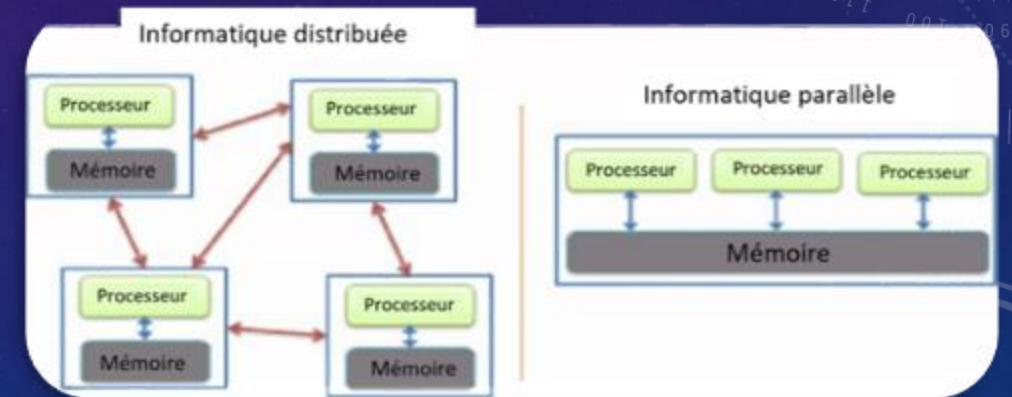
- **Calculs arithmétiques à l'aide du Map Reduce**
- **WordCount avec le paradigme du Map Reduce**

## Qu'est-ce qu'un système distribué ?

- Un système distribué est un ensemble d'ordinateurs indépendants
- Ces ordinateurs coopèrent pour apparaître comme un système unique et cohérent
- Communication par réseau, synchronisation, cohérence des données

Le **traitement parallèle** consiste à effectuer des tâches de calcul sur plusieurs processeurs à la fois afin d'améliorer la vitesse et l'efficacité du calcul. Les tâches sont divisées en sous-tâches et exécutées simultanément par différents processeurs.

Le **traitement distribué** est un processus qui consiste à connecter plusieurs ordinateurs via un réseau local ou un réseau étendu afin qu'ils puissent agir ensemble comme un seul ordinateur très puissant, capable d'effectuer des calculs qu'aucun ordinateur du réseau ne pourrait réaliser seul.



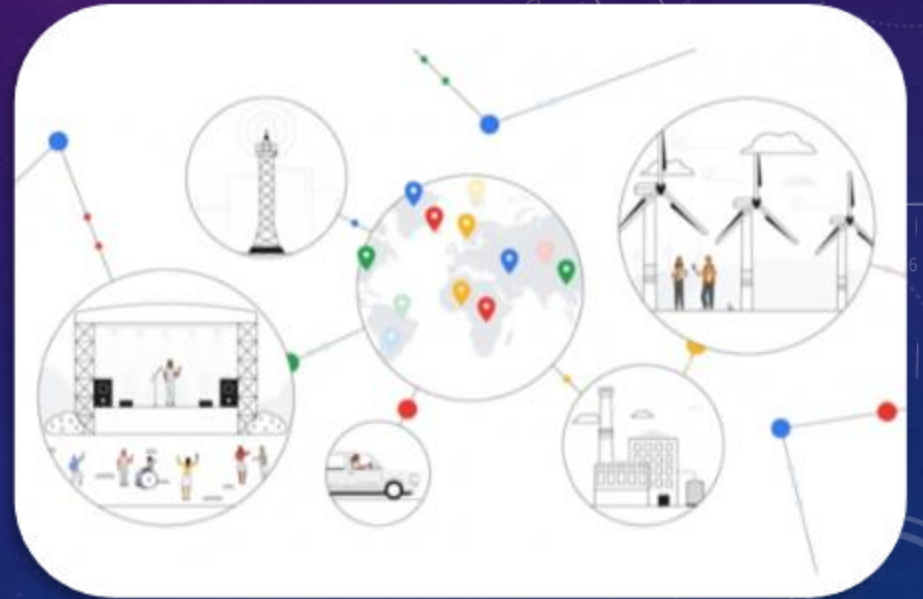
Notez bien la différence entre un traitement distribué et un système distribué

## Caractéristiques d'un système distribué

- Scalabilité : capacité à gérer l'augmentation du volume de données
- Tolérance aux pannes : le système continue à fonctionner malgré la présence de défaillances
- Transparence : l'utilisateur ne perçoit pas la distribution du traitement
- Concurrence : traitements parallèles sur plusieurs nœuds

Nous pouvons citer en exemple Google qui :

- Est capable de gérer l'indexation de milliards de pages web
- Malgré une défaillance le moteur de recherche continue à fonctionner
- Traite des requêtes en quelques millisecondes sans que l'utilisateur sache comment les données sont distribuées
- Utilise de manière massive les clusters distribués



 **Plan sur 5 séances de 3 heures**

## **Séance 1 - Introduction aux systèmes distribués et à Hadoop (3h)**

### **Théorie (1h30)**

- **Notions de base des systèmes distribués**
  - Définition, enjeux, caractéristiques
  - Scalabilité, tolérance aux pannes, latence
- **Le modèle MapReduce**
- **Présentation de l'écosystème Hadoop :**
  - Stockage distribué : HDFS
  - Calcul distribué

### **Pratique (1h30)**

- **Calculs arithmétiques à l'aide du Map Reduce**
- **WordCount avec le paradigme du Map Reduce**

## Le modèle MapReduce

- Inspiré de la programmation fonctionnelle
- Deux phases principales :
  - Map : transformation des données
  - Reduce : agrégation des résultats
- Exécution distribuée automatique



→ Article proposé en 2004 par deux chercheurs de Google :

<https://static.googleusercontent.com/media/research.google.com/fr//archive/mapreduce-osdi04.pdf>

 **Plan sur 5 séances de 3 heures**

## **Séance 1 - Introduction aux systèmes distribués et à Hadoop (3h)**

### **Théorie (1h30)**

- **Notions de base des systèmes distribués**
  - Définition, enjeux, caractéristiques
  - Scalabilité, tolérance aux pannes, latence
- **Le modèle MapReduce**
- **Présentation de l'écosystème Hadoop :**
  - Stockage distribué : HDFS
  - Calcul distribué

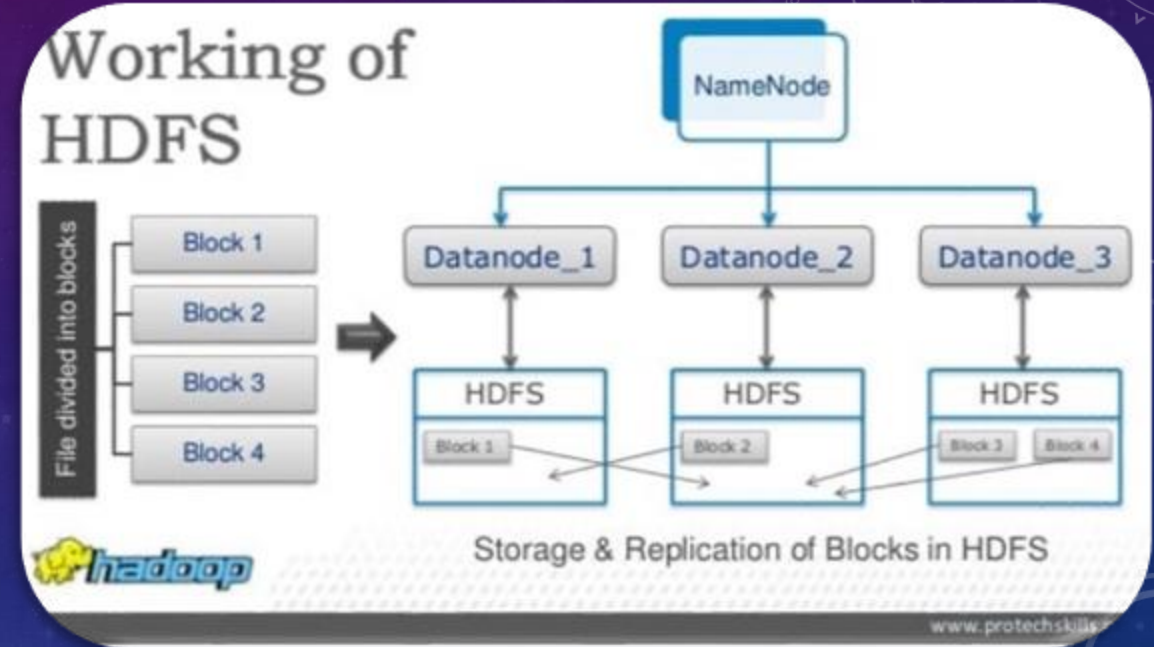
### **Pratique (1h30)**

- **Calculs arithmétiques à l'aide du Map Reduce**
- **WordCount avec le paradigme du Map Reduce**

## Présentation de Hadoop

### Stockage distribué : HDFS (Hadoop Distributed File System) :

- Présente de nombreuses similitudes avec les systèmes de fichiers distribués existants
- Très tolérant aux pannes
- Conçu pour être déployé sur du matériel à faible coût
- Accès à haut débit aux données d'application
- Convient aux applications ayant de grands ensembles de données

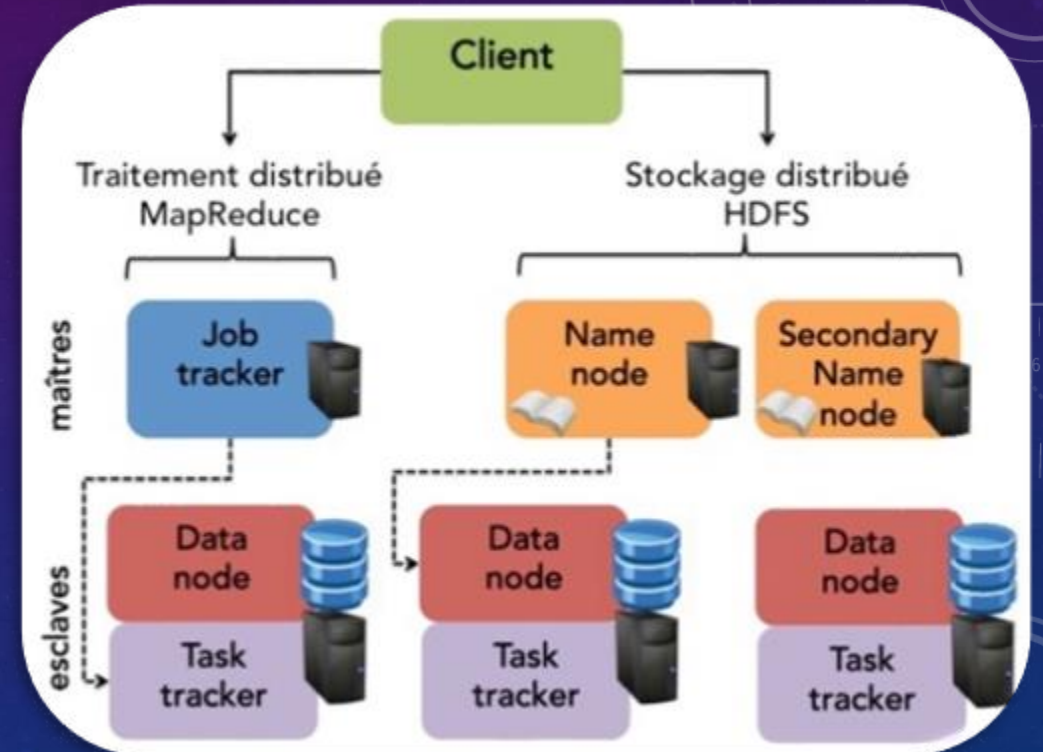


## Présentation de Hadoop

### Calcul distribué :

Le socle technique de Hadoop est composé :

- De toute l'architecture support nécessaire pour l'orchestration de MapReduce, c'est-à-dire :
  - l'ordonnancement des traitements,
  - la localisation des fichiers,
  - la distribution de l'exécution.
- D'un système de fichiers HDFS qui est :
  - Distribué : les données sont réparties sur les machines du cluster.
  - Répliqué : en cas de panne, aucune donnée n'est perdue.
  - Optimisé pour la colocalisation des données et des traitements.



 **Plan sur 5 séances de 3 heures**

## **Séance 1 - Introduction aux systèmes distribués et à Hadoop (3h)**

### **Théorie (1h30)**

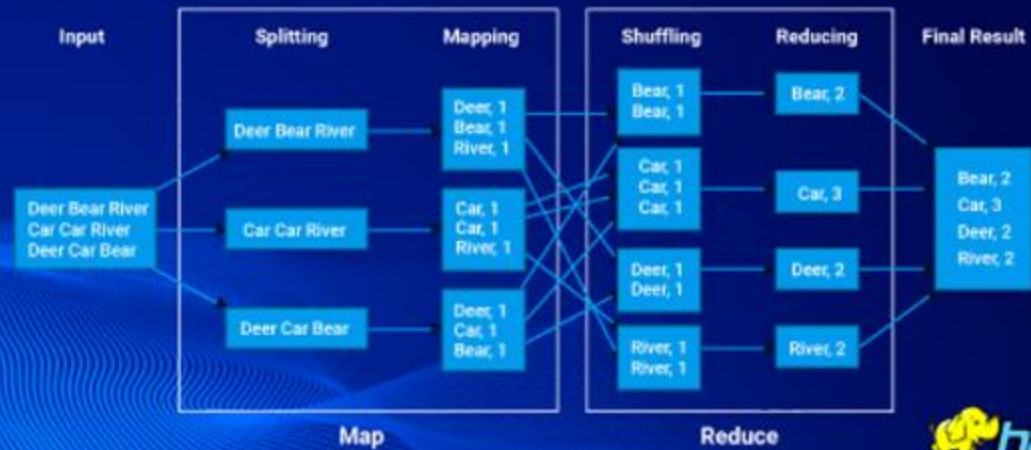
- **Notions de base des systèmes distribués**
  - Définition, enjeux, caractéristiques
  - Scalabilité, tolérance aux pannes, latence
- **Le modèle MapReduce**
- **Présentation de l'écosystème Hadoop :**
  - Stockage distribué : HDFS
  - Calcul distribué

### **Pratique (1h30)**

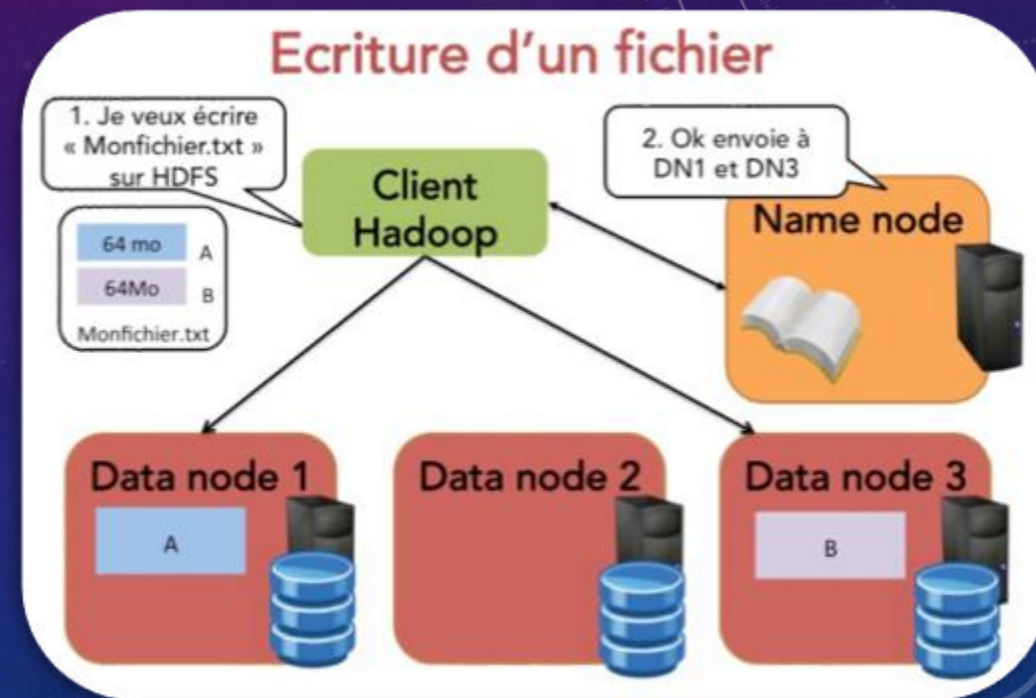
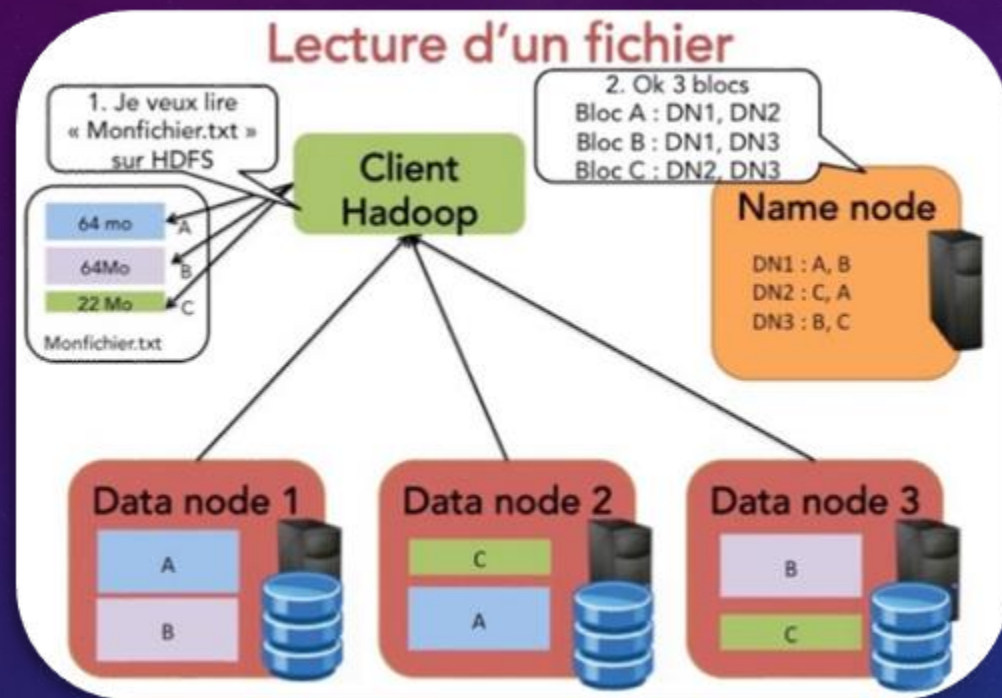
- **WordCount avec le paradigme du Map Reduce**
- **Diagramme d'architecture d'écriture et de lecture sur HDFS**

## WordCount avec Map Reduce

### UNDERSTANDING MAPREDUCE IN HADOOP



## Diagramme d'architecture d'écriture et de lecture sur HDFS



 **Plan sur 5 séances de 3 heures**

## **Séance 2 - Limitations de MapReduce et introduction à Apache Spark (3h)**

### **Théorie (1h30)**

- **Limitations de MapReduce (latence, complexité de développement)**
- **Architecture de Spark :**
  - **RDD (Resilient Distributed Dataset)**
  - **DAG (Directed Acyclic Graph)**
  - **SparkContext, Driver, Executor**
- **Spark vs Hadoop MapReduce**

### **Pratique (1h30)**

- **Introduction à PySpark ou Spark avec Scala**
- **Création d'un DataFrame, transformations & actions simples**
- **Traitement d'un DataFrame CSV localement**

 Plan sur 5 séances de 3 heures

## Séance 2 - Limitations de MapReduce et introduction à Apache Spark (3h)

### Théorie (1h30)

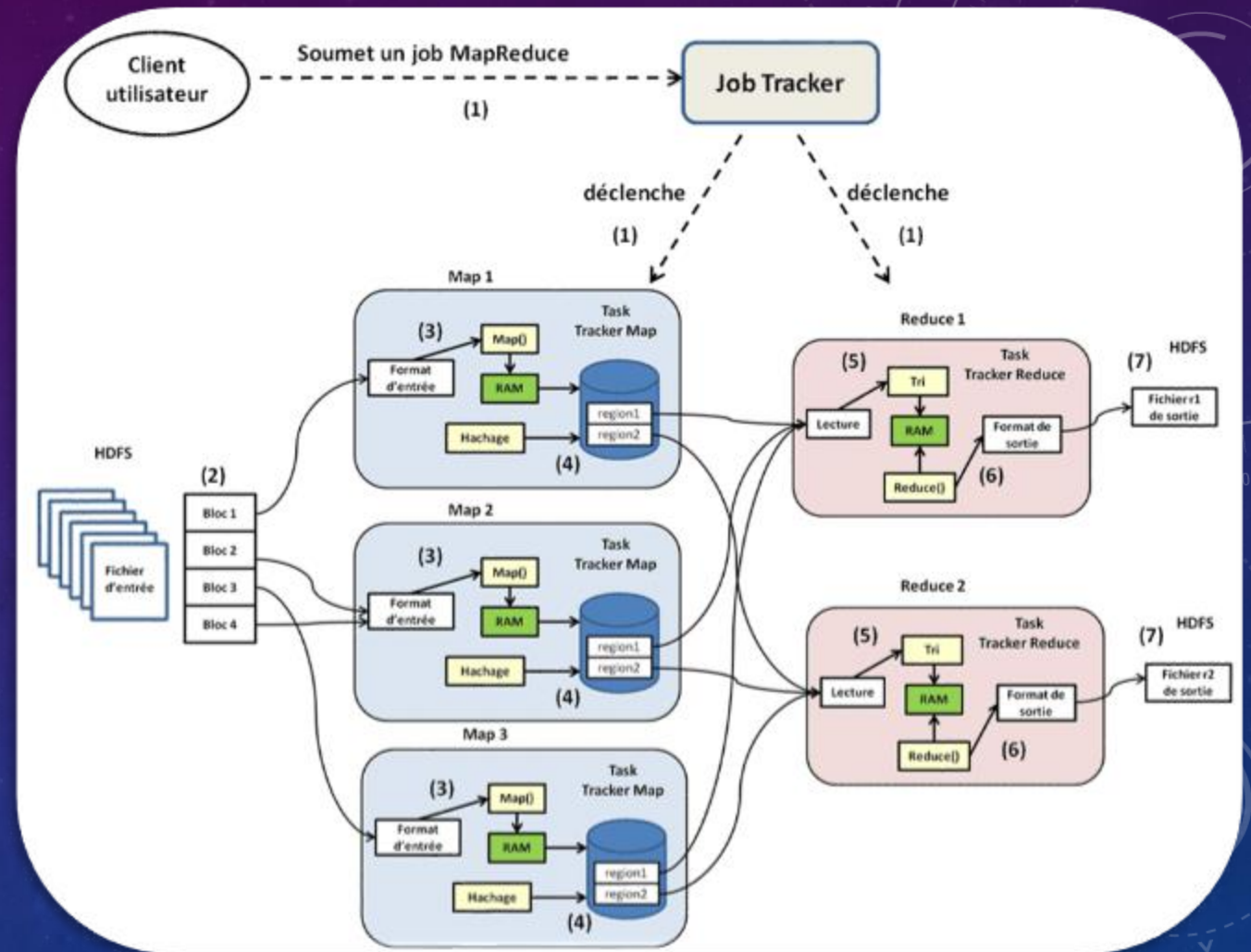
- **Limitations de MapReduce (latence, complexité de développement)**
- Architecture de Spark :
  - RDD (Resilient Distributed Dataset)
  - DAG (Directed Acyclic Graph)
  - SparkContext, Driver, Executor
- Spark vs Hadoop MapReduce

### Pratique (1h30)

- Introduction à PySpark ou Spark avec Scala
- Création d'un DataFrame, transformations & actions simples
- Traitement d'un DataFrame CSV localement

## Le modèle MapReduce

- Architecture simple mais limitée
- Lent à cause des accès disque entre chaque phase
- Complexité de développement
- Pas adapté aux traitements interactifs ou en temps réel



 **Plan sur 5 séances de 3 heures**

## **Séance 2 - Limitations de MapReduce et introduction à Apache Spark (3h)**

### **Théorie (1h30)**

- Limitations de MapReduce (latence, complexité de développement)
- **Architecture de Spark :**
  - RDD (Resilient Distributed Dataset)
  - DAG (Directed Acyclic Graph)
  - SparkContext, Driver, Executor
- Spark vs Hadoop MapReduce

### **Pratique (1h30)**

- Introduction à PySpark ou Spark avec Scala
- Création d'un DataFrame, transformations & actions simples
- Traitement d'un DataFrame CSV localement

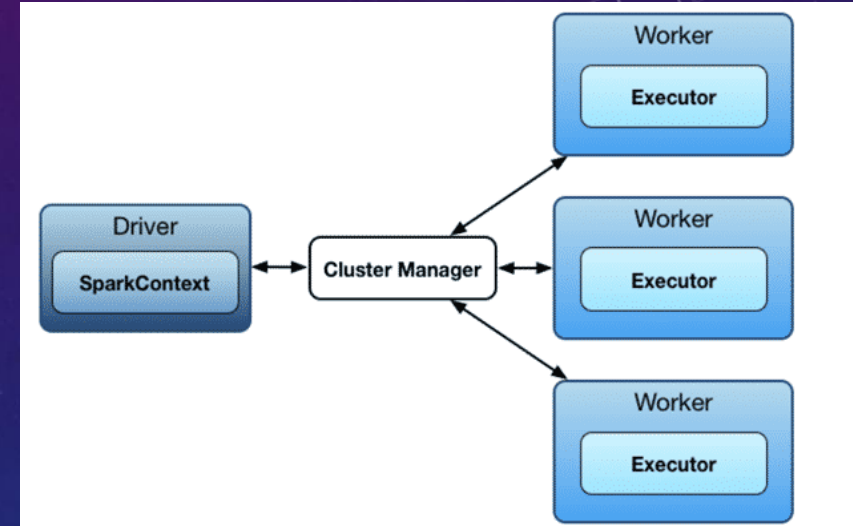
## Architecture de Spark

Pourquoi Apache Spark ?

- Traitement en mémoire
- Plus rapide qu'un Map Reduce (jusqu'à 100x)
- API plus simple (RDD, DataFrames, SQL)
- Support du batch, du streaming, du ML, ...

Éléments de Spark :

- Spark Driver : point d'entrée de l'application
- Executors/Workers : processus répartis sur les nœuds
- Cluster Manager : gère les ressources (YARN)
- DAG Scheduler : exécute un graph de tâches optimisé

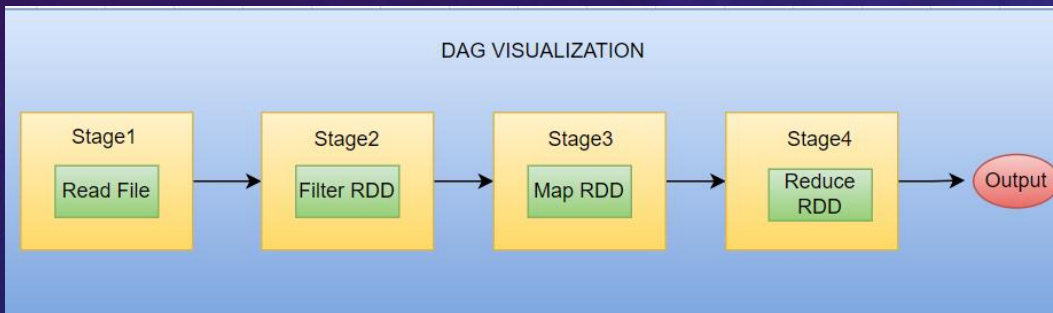


## Définitions

### **RDD** : Resilient Distributed Datasets

- Abstraction fondamentale de Spark
- Collection distribuée d'objets
- Transformations (map, filter, etc.)
- Actions (collect, count, reduce, etc.)
- Tolérance aux pannes

### **DAG** : Directed Acyclic Graph



### **Spark Context**

- Il sert de point d'entrée pour interagir avec une application Spark 1.x.
- Il se connecte au cluster Spark et sert de programme pilote pour soumettre les tâches Spark.
- Il gère les ressources du cluster, fonctionnant à la fois comme gestionnaire de cluster et gestionnaire de ressources pour l'application

 **Plan sur 5 séances de 3 heures**

## **Séance 2 - Limitations de MapReduce et introduction à Apache Spark (3h)**

### **Théorie (1h30)**

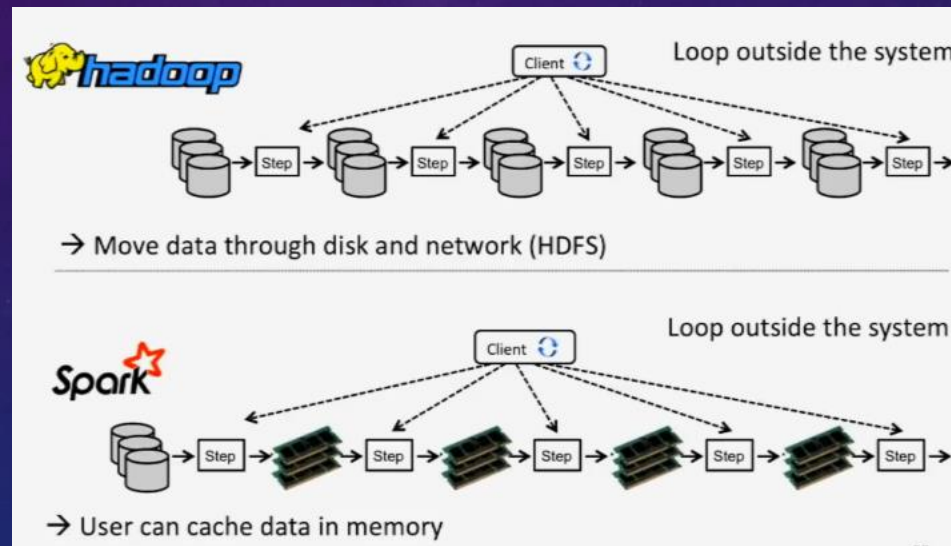
- **Limitations de MapReduce (latence, complexité de développement)**
- **Architecture de Spark :**
  - **RDD (Resilient Distributed Dataset)**
  - **DAG (Directed Acyclic Graph)**
  - **SparkContext, Driver, Executor**
- **Spark vs Hadoop MapReduce**

### **Pratique (1h30)**

- **Introduction à PySpark ou Spark avec Scala**
- **Création d'un DataFrame, transformations & actions simples**
- **Traitement d'un DataFrame CSV localement**

## Spark vs Hadoop MapReduce

- Spark stocke les données en mémoire entre les étapes
- Moins d'I/O disque, donc plus rapide
- API haut niveau (Python, Scala, Java, SQL)



 **Plan sur 5 séances de 3 heures**

## **Séance 2 - Limitations de MapReduce et introduction à Apache Spark (3h)**

### **Théorie (1h30)**

- **Limitations de MapReduce (latence, complexité de développement)**
- **Architecture de Spark :**
  - **RDD (Resilient Distributed Dataset)**
  - **DAG (Directed Acyclic Graph)**
  - **SparkContext, Driver, Executor**
- **Spark vs Hadoop MapReduce**

### **Pratique (1h30)**

- **Introduction à PySpark ou Spark avec Scala**
- **Création d'un DataFrame, transformations & actions simples**
- **Traitement d'un DataFrame CSV localement**

 **Plan sur 5 séances de 3 heures**

## **Séance 3 - Traitement structuré avec Spark SQL et DataFrames (3h)**

### **Théorie (1h30)**

- **Avantages des DataFrames par rapport aux RDD**
- **Spark SQL**
  - **Schémas, requêtes, intégration SQL**
  - **Optimisation via le Catalyst Optimizer**
- **Formats de données : CSV, JSON, Parquet, Avro**

### **Pratique (1h30)**

- **Requêtes SQL sur un DataFrame en Spark**
- **Chargement et analyse d'un DataFrame structuré (ex: OpenData)**
- **Exploration et visualisation simple des résultats**

 **Plan sur 5 séances de 3 heures**

## **Séance 3 - Traitement structuré avec Spark SQL et DataFrames (3h)**

### **Théorie (1h30)**

- **Avantages des DataFrames par rapport aux RDD**
- **Spark SQL**
  - Schémas, requêtes, intégration SQL
  - Optimisation via le Catalyst Optimizer
- **Formats de données : CSV, JSON, Parquet, Avro**

### **Pratique (1h30)**

- **Requêtes SQL sur un DataFrame en Spark**
- **Chargement et analyse d'un DataFrame structuré (ex: OpenData)**
- **Exploration et visualisation simple des résultats**

## Avantages des DataFrames par rapport aux RDD

Limite des RDDs :

- Pas de schéma implicite
- Optimisations limitées
- Moins adapté à l'analyse de données structurées

Qu'est ce qu'un DataFrame ?

- Structure tabulaire avec schéma (colonnes nommées)
- Inspiré des DataFrames de R / Python
- API haut niveau pour les transformations
- Compatible avec plusieurs langages (Python, Scala, Java, R)

 Plan sur 5 séances de 3 heures

## Séance 3 - Traitement structuré avec Spark SQL et DataFrames (3h)

### Théorie (1h30)

- Avantages des DataFrames par rapport aux RDD
- **Spark SQL**
  - Schémas, requêtes, intégration SQL
  - Optimisation via le Catalyst Optimizer
- Formats de données : CSV, JSON, Parquet, Avro

### Pratique (1h30)

- Requetes SQL sur un DataFrame en Spark
- Chargement et analyse d'un DataFrame structuré (ex: OpenData)
- Exploration et visualisation simple des résultats

## Spark SQL

### Qu'est ce que c'est ?

- Permet d'exécuter des requêtes SQL sur des DataFrames
- Intègre un moteur d'optimisation : Catalyst
- Peut lire des données depuis HDFS, Hive, JDBC, etc.
- Supporte les vues temporaires et permanentes

### Optimisation :

- Catalyst : planification logique et physique des requêtes
- Résolution des noms de colonnes, type checking
- Gestion du traitement in memory

 **Plan sur 5 séances de 3 heures**

## **Séance 3 - Traitement structuré avec Spark SQL et DataFrames (3h)**

### **Théorie (1h30)**

- **Avantages des DataFrames par rapport aux RDD**
- **Spark SQL**
  - **Schémas, requêtes, intégration SQL**
  - **Optimisation via le Catalyst Optimizer**
- **Formats de données : CSV, JSON, Parquet, Avro**

### **Pratique (1h30)**

- **Requêtes SQL sur un DataFrame en Spark**
- **Chargement et analyse d'un DataFrame structuré (ex: OpenData)**
- **Exploration et visualisation simple des résultats**

## Formats de données

Formats supportés : CSV, JSON, Parquet, Avro, ...

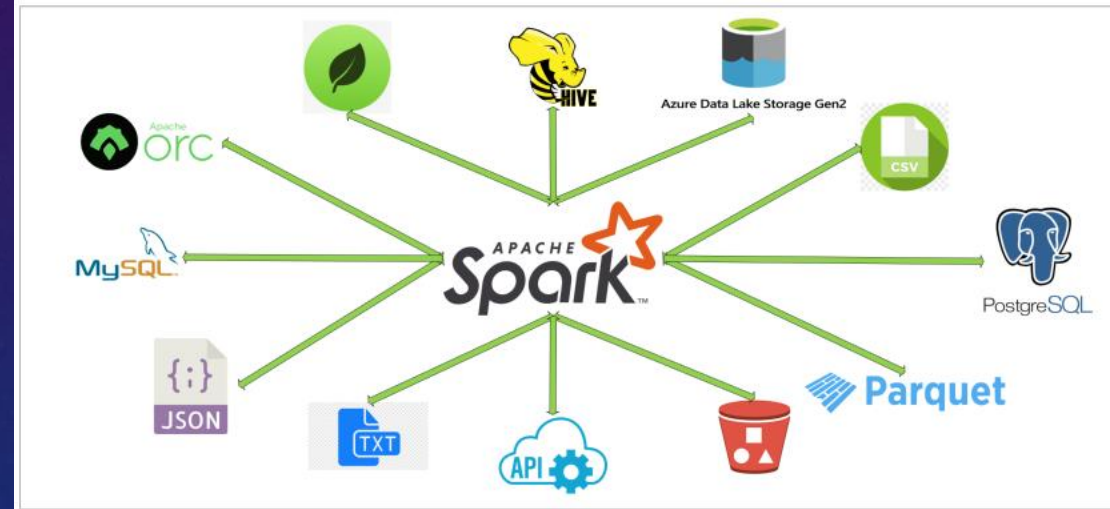
Exemples :

Lecture :

```
df = spark.read.csv("fichier.csv", header=True, inferSchema=True)
```

Écriture :

```
df.write.parquet("output.parquet")
```



 **Plan sur 5 séances de 3 heures**

## **Séance 3 - Traitement structuré avec Spark SQL et DataFrames (3h)**

### **Théorie (1h30)**

- **Avantages des DataFrames par rapport aux RDD**
- **Spark SQL**
  - **Schémas, requêtes, intégration SQL**
  - **Optimisation via le Catalyst Optimizer**
- **Formats de données : CSV, JSON, Parquet, Avro**

### **Pratique (1h30)**

- **Requêtes SQL sur un DataFrame en Spark**
- **Chargement et analyse d'un DataFrame structuré (ex: OpenData)**
- **Exploration et visualisation simple des résultats**

 **Plan sur 5 séances de 3 heures**

## **Séance 4 - Traitement temps réel avec Spark Streaming (3h)**

### **Théorie (1h30)**

- **Traitement batch vs streaming**
- **Spark Streaming :**
  - **Micro-batching, DStream vs Structured Streaming**
  - **Intégration avec Kafka**

### **Pratique (1h30)**

- **Simulation d'un flux (socket)**
- **Traitement de flux en direct avec Spark**
- **Word Count en streaming avec Spark**

 Plan sur 5 séances de 3 heures

## Séance 4 - Traitement temps réel avec Spark Streaming (3h)

### Théorie (1h30)

- **Traitement batch vs streaming**
- Spark Streaming :
  - Micro-batching, DStream vs Structured Streaming
  - Intégration avec Kafka

### Pratique (1h30)

- Simulation d'un flux (socket)
- Traitement de flux en direct avec Spark
- Word Count en streaming avec Spark

## Traitement batch vs streaming

Qu'est ce que le streaming ?

- Traitement de données en temps réel ou quasi réel
- Données continues : capteurs, logs, événements utilisateurs...
- Contraintes : latence, volume, fiabilité



 Plan sur 5 séances de 3 heures

## Séance 4 - Traitement temps réel avec Spark Streaming (3h)

### Théorie (1h30)

- Traitement batch vs streaming
- **Spark Streaming** :
  - Micro-batching, DStream vs Structured Streaming
  - Intégration avec Kafka

### Pratique (1h30)

- Simulation d'un flux (socket)
- Traitement de flux en direct avec Spark
- Word Count en streaming avec Spark

 **Plan sur 5 séances de 3 heures**

## **Séance 4 - Traitement temps réel avec Spark Streaming (3h)**

### **Théorie (1h30)**

- **Traitement batch vs streaming**
- **Spark Streaming :**
  - **Micro-batching, DStream vs Structured Streaming**
  - **Intégration avec Kafka**

### **Pratique (1h30)**

- **Simulation d'un flux (socket)**
- **Traitement de flux en direct avec Spark**
- **Word Count en streaming avec Spark**

 **Plan sur 5 séances de 3 heures**

## **Séance 5 - Projet pratique et présentations orales (3h)**

### **Projet (2h)**

- **Mini-projet de traitement distribué avec Spark**
- **Analyse de logs, traitement de fichiers volumineux**
- **Ou traitement de flux avec restitution simple**
- **Travail en groupes**

### **Présentations orales (1h)**

- **Q&R, évaluation, feedback**

 **Plan sur 5 séances de 3 heures**

## **Séance 5 - Projet pratique et présentations orales (3h)**

### **Projet (2h)**

- **Mini-projet de traitement distribué avec Spark**
- **Analyse de logs, traitement de fichiers volumineux**
- **Ou traitement de flux avec restitution simple**
- **Travail en groupes**

### **Présentations orales (1h)**

- **Q&R, évaluation, feedback**

 **Plan sur 5 séances de 3 heures**

## **Séance 5 - Projet pratique et présentations orales (3h)**

### **Projet (2h)**

- **Mini-projet de traitement distribué avec Spark**
- **Analyse de logs, traitement de fichiers volumineux**
- **Ou traitement de flux avec restitution simple**
- **Travail en groupes**

### **Présentations orales (1h)**

- **Q&R, évaluation, feedback**